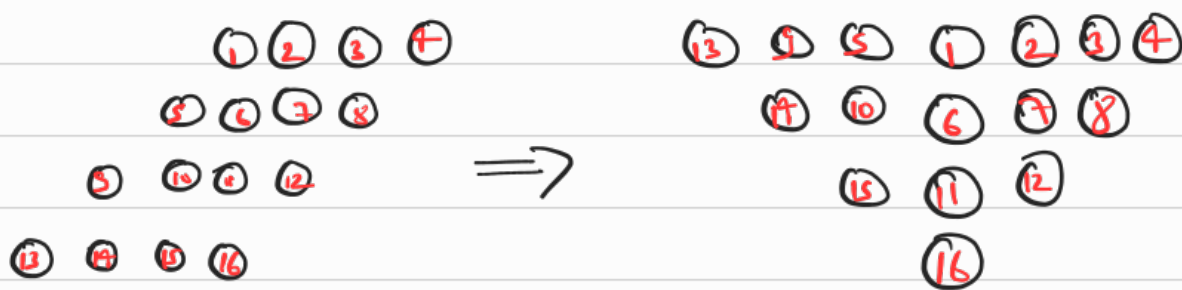


FA $\Rightarrow$ 3 $\rightarrow$ 2 compressor input 3 bits, out 2 bits

HA $\Rightarrow$ 2 $\rightarrow$ 2 compressor input 2 bits out, 2 bits

Wallace Tree :

partial products

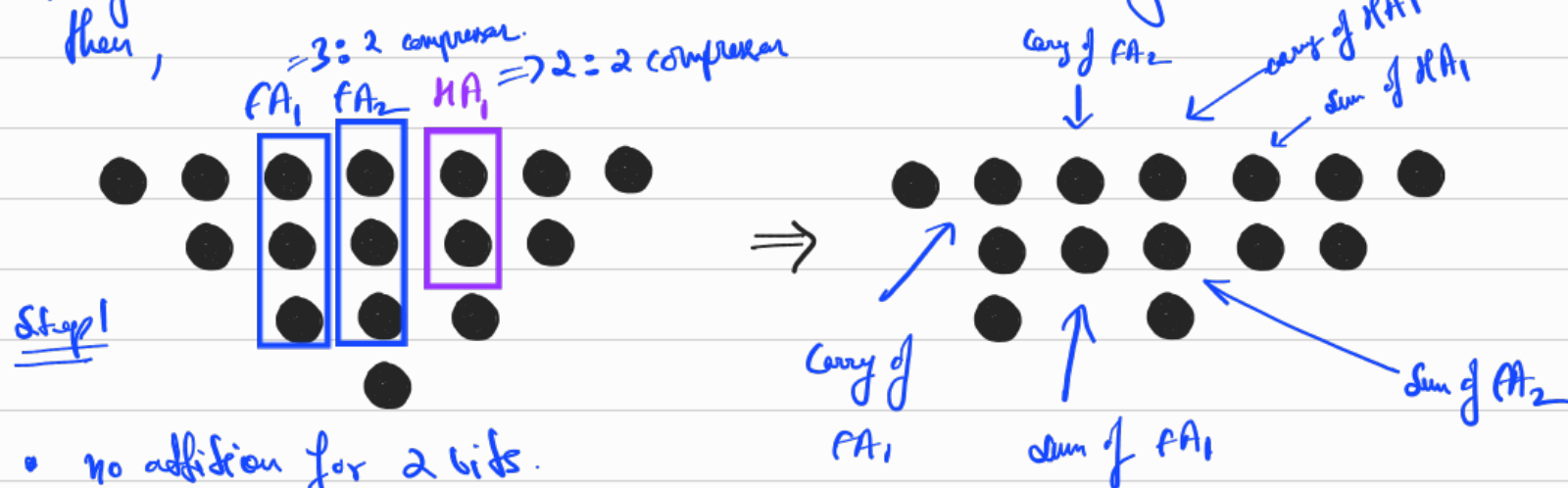


- We only use FA & HA to calculate the whole.
- now we will only represent here as

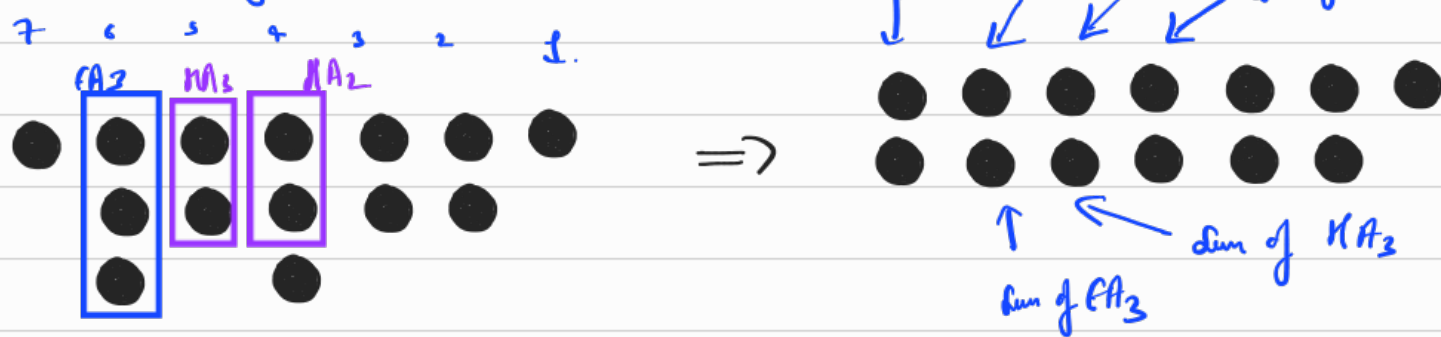
$\hookrightarrow$ then with the help of FA & HA we reduce the terms.

i) now to reduce, we will only use FA for same bit position only when the bits before it (HdB) has more than 2 bits.

ii) for 2 bit addition we don't use HA right now then,



- no addition for 2 bits.
- HA used only if previous bits are ≤ 2 .
- FA used only when prev bits > 2 ...



Step 2 : Repeat.

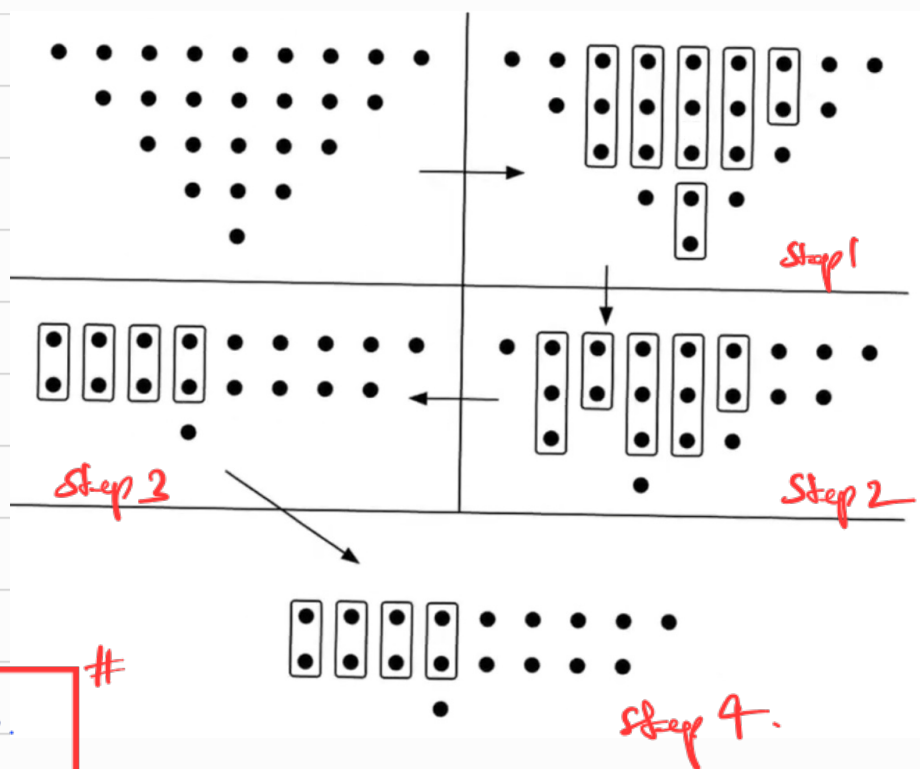
- at bit position 5 HA is used since that's the max we can cover. 2-bits previous bits had more than 2 then we used.
- we don't use HA for 2 bits if seeing cont. 2 bits from LdB.

Now we can feed this directly to a fast adder.

- we avoided use of HA & FA for whole as these terms might be redundant.

then another example of 5 bit adder is

- The delay of the multiplier is dependent only on the number of steps used which is tough to find.



we have

$$r_{j+1} = 2 \cdot \text{floor}\left(\frac{r_j}{2}\right) + r_j \bmod 3 \quad \#$$

here r_0 = no. of bits in multiplier.
 & we stop when $r_n = 2$.

Ex. for a 6×6 multiplier we have,
 $r_0 = 6$

$$r_1 = 2 \cdot \text{floor}\left(\frac{6}{2}\right) + 6 \bmod 3 \Rightarrow 4.$$

$$r_2 = 2 \cdot \text{floor}\left(\frac{4}{2}\right) + 4 \bmod 3 \Rightarrow 3$$

$$r_3 = 2 \cdot \text{floor}\left(\frac{3}{2}\right) + 3 \bmod 3 \Rightarrow 2 \quad] \text{ Stop.}$$

3 steps required to reduce to 2 bit combo.

- Dadda multiplier:

$$d_{n+1} = \text{floor}\left(\frac{3 \cdot d_n}{2}\right) \quad \text{where } d_1 = 2.$$

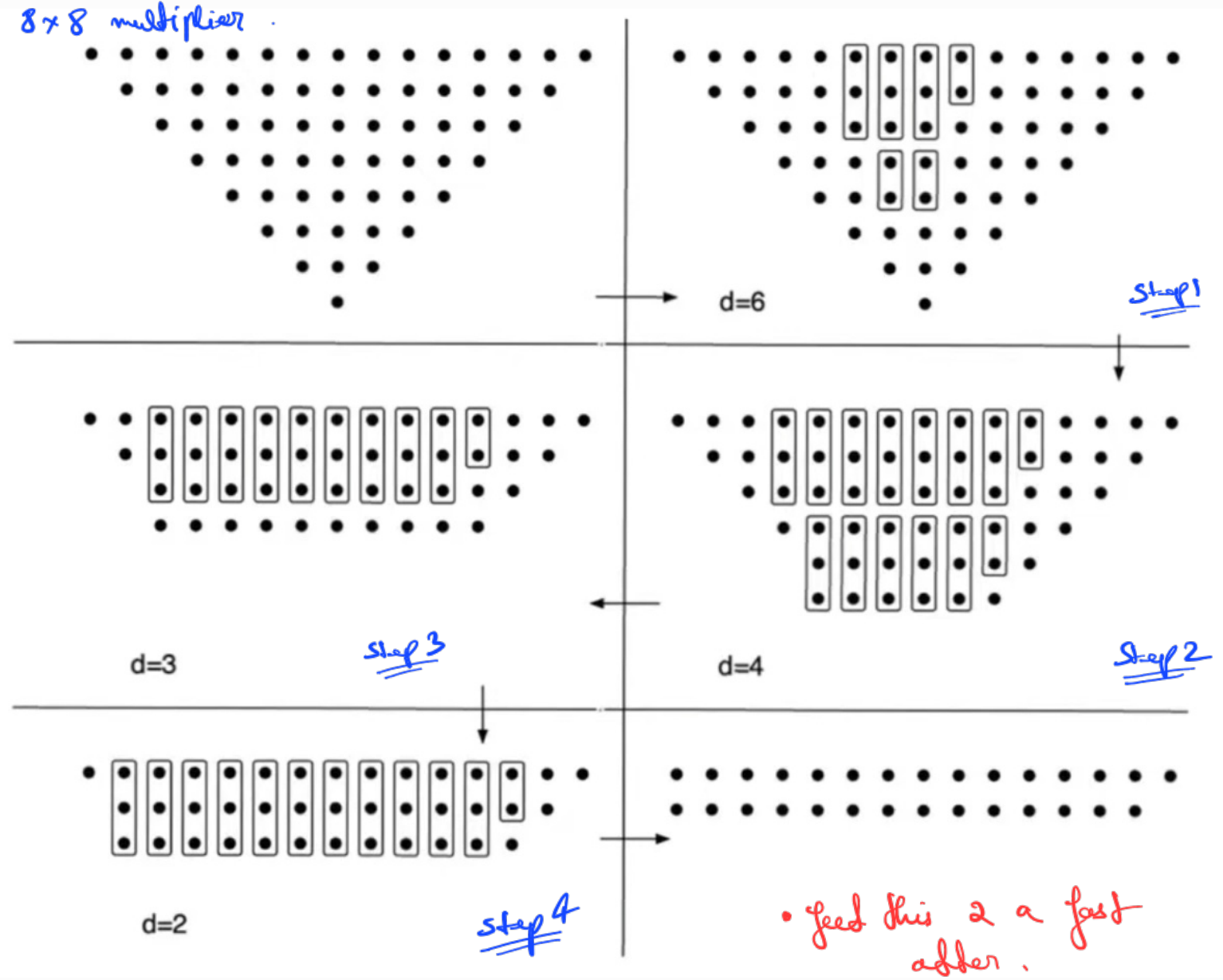
thus : 2, 3, 4, 6, 9, 13, 19, 28, ...

thus for a $M \times N$ multiplier we have,
 $d < \min(M, N)$

thus for 8×9 we have $d < \min(8, 9)$
 $\Rightarrow d = 6$ (as per above sequence).

- now, from WB we begin & start reducing the combo to d -bits like we did it above for 6 bits.
- we can use FA, PFA to reduce the bits.
- since we have d bits combo & then we move to $d_{\text{new}} < d_{\text{old}} \Rightarrow$ then same reduction until $d = 2$.

8x8 multiplier.



total steps $\Rightarrow 4$.

now if we had Wallace tree multiplier then:

$$r_0 = 8$$

$$r_1 = 2 \cdot \text{floor}\left(\frac{8}{3}\right) + 8 \bmod 3 \Rightarrow 6$$

$$r_2 = 2 \cdot \text{floor}\left(\frac{6}{3}\right) + 6 \bmod 3 \Rightarrow 4$$

$$r_3 = 2 \cdot \text{floor}\left(\frac{4}{3}\right) + 4 \bmod 3 \Rightarrow 3$$

$$r_4 = 2 \cdot \text{floor}\left(\frac{3}{3}\right) + 3 \bmod 3 \Rightarrow 2$$

• here also 4 steps.

• Carry Save Adder:

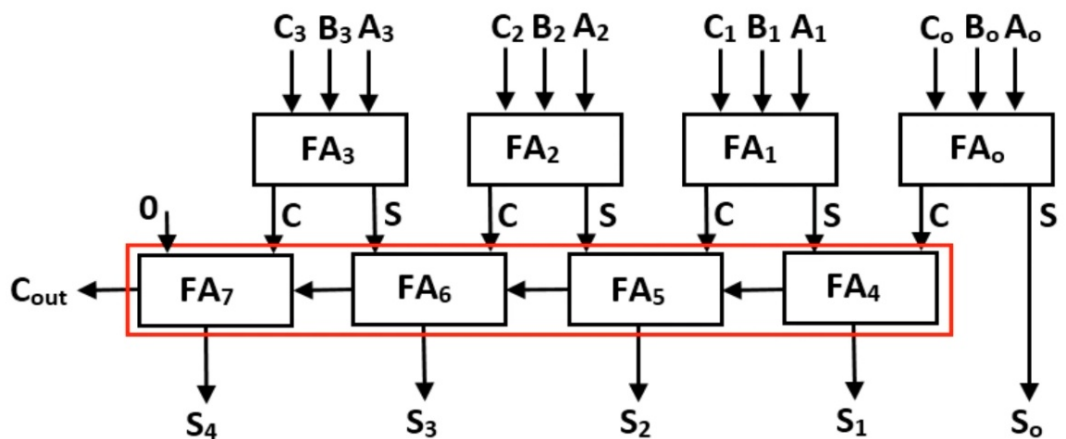
• good for addition of more than 2 numbers.

suppose
 $A + B + C$

then,
 $(A+B) + C$

$\Rightarrow \hookrightarrow 2 \text{ RCA}$

thus $2 \times \text{RCA delay}$.



• In CSA $\Rightarrow$ only one RCA thus single delay chain.

Concept of CSA.

$$\begin{array}{r}
 X \quad 10101 \\
 Y \quad + 10011 \\
 Z \quad + 01011 \\
 \hline
 C \quad 10011
 \end{array}$$

Carry $\Rightarrow$ left shift by 1 then,
100110

now add sum & carry.

$$\begin{array}{r}
 S \quad 01101 \\
 C \quad 100110 \\
 \hline
 \text{final} \quad 110011
 \end{array}$$

- CSA is used in reduction steps of Wallace & Dadda, basically it's just FA & HA but the FA output is not fed back as Cin rather as a input.

$\hookrightarrow$ Just a connection difference nothing else.

$\Rightarrow$ Dadda multiplier basically is basically a algorithm or can say methodology.

now since the best possible reduction is $3 \rightarrow 2$ (FA)

thus $\frac{3}{2} \Rightarrow 1.5$

hence we have the formula as

$$2d+1 = \text{floor}\left(\frac{3}{2} * du\right)$$

Imp

Feature	Wallace	Dadda
Speed	🔥 Fastest	⚡ Very fast
Area	✗ High	✓ Low
Power	✗ High	✓ Low
Design complexity	Medium	High
Preferred for	Max-speed designs	Balanced PPA designs (most ASICs)



less over compression in a single step.



reduction steps can be more

- fast adders: faster than ripple carry.
 - $\hookrightarrow$ parallel carry computation
 - $\hookrightarrow$ carry lookahead
 - $\hookrightarrow$ carry bypass
 - $\hookrightarrow$ prefix trees
 - $\hookrightarrow$ carry selection.

CLA (Carry lookahead Adder)

A	B	C _{in}	C _o
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

thus ,

$$C_o = \underbrace{AB}_{\text{generate}} + \underbrace{(A \oplus B) \cdot C_{in}}_{\text{propagate}}$$

thus ,

$$C_o = G + PC_{in}$$

thus ,

$$C_i = G_i + P_i C_{i-1}$$

thus say

$$C_o = G_o + P_o \cdot C_{-1}$$

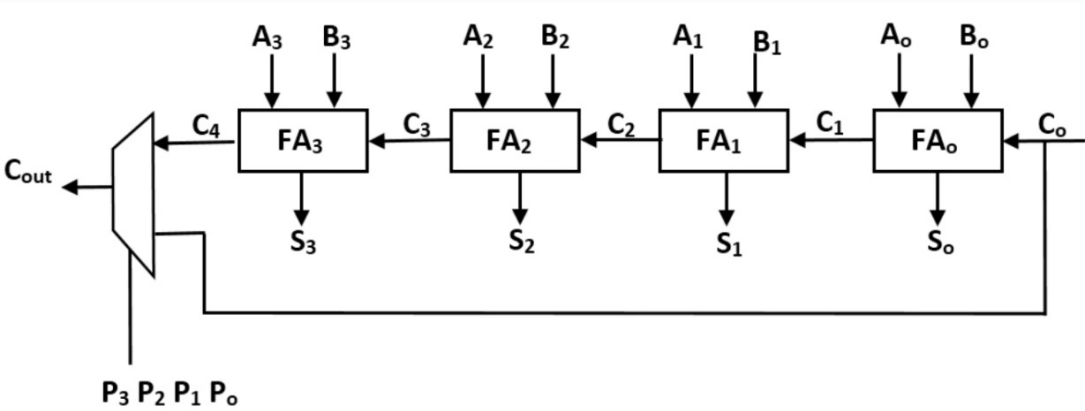
then ,

$$C_1 = G_1 + P_1 G_o + P_1 P_o C_{-1}$$

then ,

$$C_2 = G_2 + P_2 G_1 + P_2 P_1 G_o + P_2 P_1 P_o C_{-1}$$

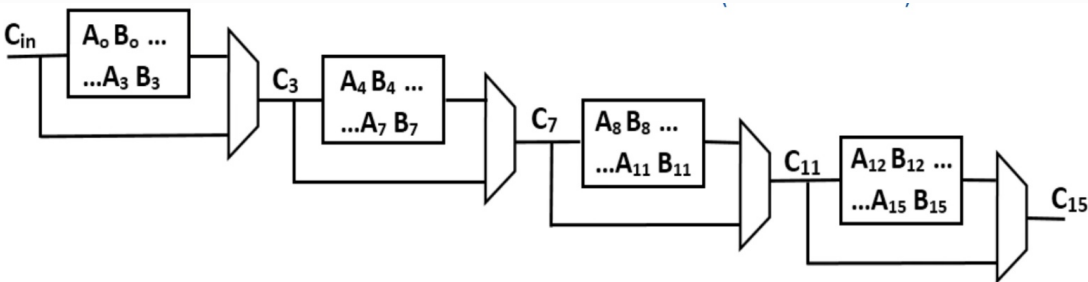
- limiting factor $\Rightarrow$ fan out.
 - & even if divided in multiple stages then not much time saving.
 - Carry skip adder / Carry Bypass adder.
- we have already seen that carry propagates. thus on that we have ,



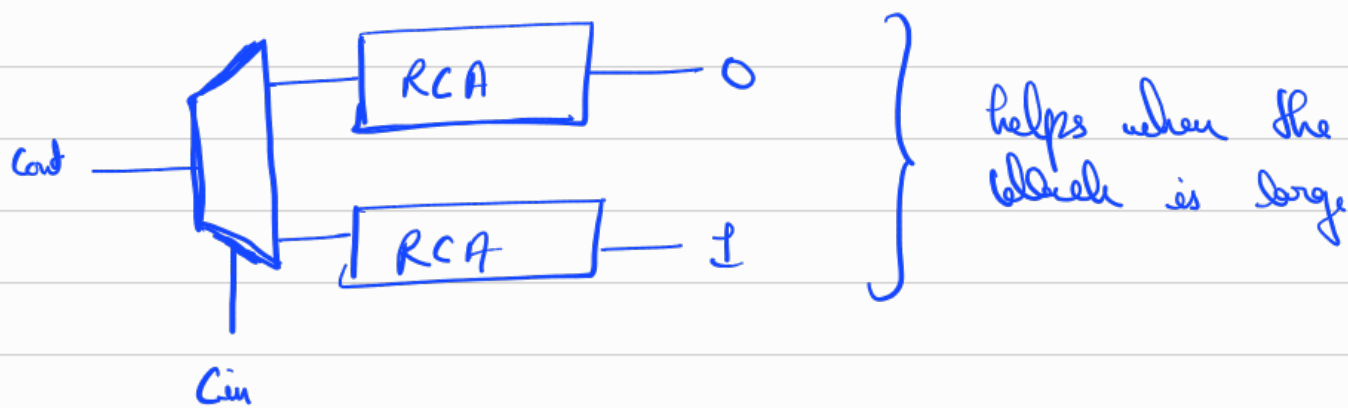
Propagate scenario: $P_i = A_i \oplus B_i = 1 \rightarrow C_{in} = C_{out}$
 Skip Condition $\rightarrow P_3 P_2 P_1 P_0 = 1$

this , we can also break it into multiple parts for large values.

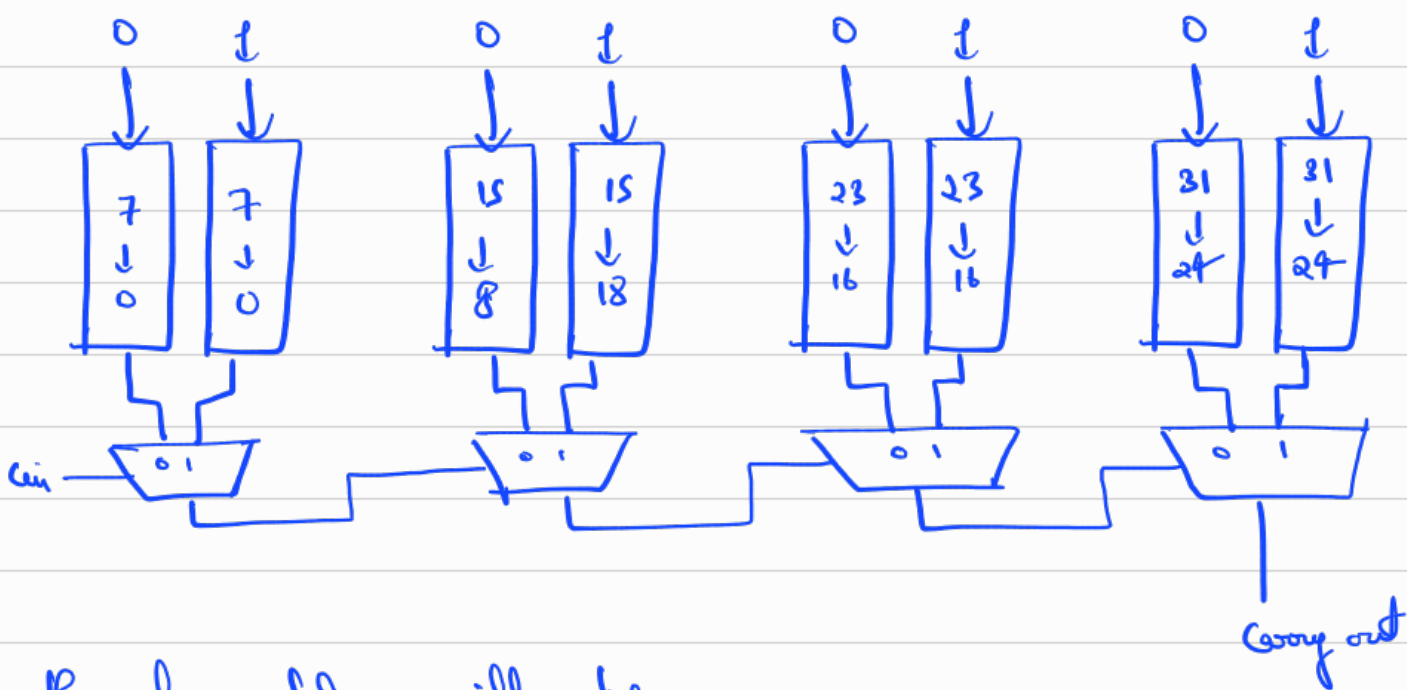
breaking as :



Carry Select Adder



example. for 32 bit assume delay of carry propagation is t_d .
 then for RCA $\Rightarrow 32t_d \leftarrow$ total delay.
 this can be reduced by carry select.



then here delay will be
 $8t_d + 4t_{mux}$
 same RCA. due to MUX